MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Done by: st. gr. FAF-242, Temciuc Adelina

Report

on laboratory work no. 3
Operating Systems
Topic: The Command Line: Shell, Pipes and Text Filters

Checked by:
Reiţman Patricia, university assistant
DISA, FCIM, UTM

Chişinău – 2026

AIM OF THE WORK

Purpose of the laboratory work

The purpose of this laboratory work is to learn to work efficiently in the bash shell, to predict what the shell does to a command line before the command is executed, to redirect the output of programs to the desired destination, and to answer real questions about data using short chains of small text filters joined by pipes.

The laboratory work aims to:

- use Tab completion, history search and the built-in documentation tools (`man`, `help`, `--help`);
- distinguish between shell built-ins and external programs and understand how the shell uses `PATH`;
- study globbing, quoting, brace expansion, arithmetic and command substitution;
- use redirection of stdout and stderr, command chaining and background jobs;
- answer questions about a log file with one-line pipelines built from `grep`, `cut`, `sort`, `uniq`, `wc`, `head` and `tail`;
- write a first bash script with argument checking, error reporting and an exit code.

THEORETICAL BACKGROUND

The shell, built-ins and PATH

The shell is a program that reads a command line, transforms it and starts the requested programs. Some commands, such as `cd` and `echo`, are built into the shell and run inside the shell process itself; all other commands are external programs that the shell looks up in the directories listed in the `PATH` variable, in order [1]. When a command cannot be found, bash returns the exit status 127 [2].

Expansion and quoting

Before running a command, the shell performs several expansions on the line: brace expansion (`{1..5}`), parameter expansion (`$HOME`), command substitution (`$(...)`), arithmetic expansion (`$((...))`), word splitting and pathname expansion, also known as globbing (`*`, `?`, `[...]`). Single quotes disable all expansions, double quotes keep parameter, command and arithmetic expansion but prevent word splitting and globbing, and a backslash quotes the single character that follows it [2].

Standard streams, redirection and pipes

Every process starts with three open file descriptors: standard input (0), standard output (1) and standard error (2). The operators `>`, `>>`, `<` and `2>` connect these descriptors to files, `2>&1` duplicates one descriptor onto another, and the pipe `|` connects the standard output of one command to the standard input of the next one. Redirections are processed from left to right [2]. Small filters such as `grep`, `cut`, `sort` and `uniq` read from standard input and write to standard output, which allows them to be combined into pipelines [3].

Shell scripts

A shell script is a text file containing shell commands. The first line, called the shebang (`#!/bin/bash`), tells the kernel which interpreter must run the file. To be executed directly, the file must have the execute permission, and the value given to `exit` becomes the exit status of the script, where 0 means success and any non-zero value means an error [2], [4].

PART 1. MOVE FAST, FIND HELP

1.1 Tab completion and history search

The text `ls /usr/sh` followed by Tab pressed twice and the text `cd /et` followed by Tab pressed once were typed in the terminal.

When Tab was pressed once and there was only one possible match, the word was completed fully on the screen (`cd /et` became `cd /etc/`). In the other case, a single press of Tab did nothing, because the name was not yet unique; pressing Tab a second time lists the possible completions.



Figure 1.1 – Tab completion

After running three different commands, Ctrl+R was pressed. A search panel appeared (`reverse-i-search`), in which the history of previously executed commands can be searched by typing a few letters; pressing Enter runs the command that was found.

1.2 Built-in and external commands, PATH and exit codes

The following commands were executed:

```
type cd ; type ls ; type echo ; which ls
echo $PATH
echo $PATH | tr ':' '\n'
nosuchprog ; echo "exit code: $?"
```

The output was:

```
cd is a shell builtin
ls is aliased to `ls --color=auto'
echo is a shell builtin
/usr/bin/ls
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:
/usr/local/games:/snap/bin:/snap/bin
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin
/sbin
/bin
/usr/games
/usr/local/games
/snap/bin
/snap/bin
nosuchprog: command not found
exit code: 127
```

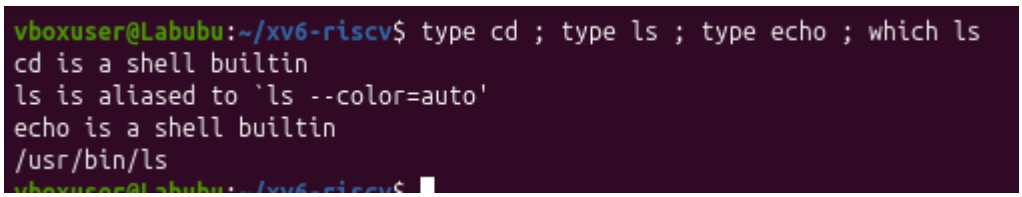


Figure 1.2 – Output of type and which



Figure 1.3 – The directories in PATH

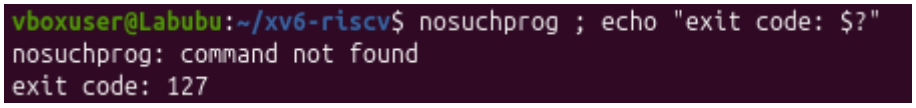


Figure 1.4 – Exit code of a missing command

1.3 Finding help without a browser

The following commands were used to find documentation:

```
man ls                    # inside: /sort then n for next, q to quit
ls --help | head
man -k "list directory"
help cd
```

Each of `man`, `help` and `--help` shows different information, depending on what is needed:

- `man ls` opens the full manual page of the `ls` program; inside it, `/sort` searches for the word "sort", `n` moves to the next match and `q` quits the manual;
- `ls --help` prints a shorter usage and options reference provided by the program itself;
- `man -k "list directory"` searches the descriptions of all manual pages and returned, among others, `ls (1)`, `dir (1)` and `vdir (1)`;
- `help cd` shows the bash built-in help for `cd`, i.e. what options and arguments can follow the `cd` command, since `cd` has no separate program and therefore no regular manual page of its own.

1.4 Answers to the questions

1. Which of cd, ls, echo are built-ins, and why must cd be one?

`cd` and `echo` are shell built-ins. `ls` is not a built-in: it is an external program located at `/usr/bin/ls` (in this shell it is also aliased to `ls --color=auto`).

`cd` must be a built-in because it has to change the current working directory of the shell process itself. If `cd` were an external program, it would run in a separate child process and would change only the directory of that process, while the directory of the shell would remain unchanged.

2. How many directories are in your PATH, and in which one is ls?

The `PATH` contains 10 entries (9 distinct directories, because `/snap/bin` appears twice): `/usr/local/sbin`, `/usr/local/bin`, `/usr/sbin`, `/usr/bin`, `/sbin`, `/bin`, `/usr/games`, `/usr/local/games`, `/snap/bin` and `/snap/bin`. The program `ls` is located in `/usr/bin`, as shown by `which ls`, which returned `/usr/bin/ls`.

3. What exit code does a command that is not found return?

A command that is not found returns the exit code 127. Running `nosuchprog ; echo "exit code: $?"` printed `nosuchprog: command not found` followed by `exit code: 127`.

4. Which option of ls sorts by modification time?

The option is `-t`. According to `man ls`, `-t` sorts by modification time (the mtime in the inode), newest first.

PART 2. WHAT THE SHELL DOES TO YOUR LINE

2.1 Preparing the test directory

A test directory with several files was created with the following commands:

```
mkdir -p ~/os-lab3/play && cd ~/os-lab3/play
touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
ls
```

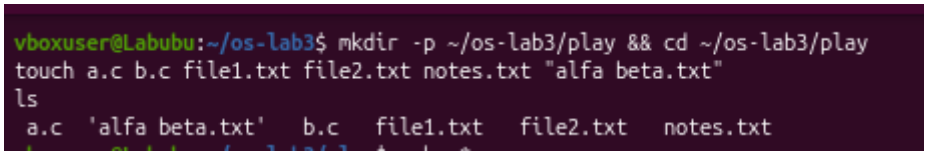


Figure 2.1 – Files created in the play directory

2.2 Predictions and real output

For each command, a prediction was written down first, then the command was run and its real output was noted. The results are presented in table 2.1.

Table 2.1 – Predictions and real output of the shell expansions

Command	My prediction	Real output	Explanation (if different)
`echo *.c`	Will show all files with the .c extension	`a.c b.c`	Matches the prediction.
`echo file?.txt`	All files that start with "file" and end with ".txt", with one changing character in place of ?	`file1.txt file2.txt`	Matches the prediction.
`echo [a-b]*`	Did not know	`a.c alfa beta.txt b.c`	[a-b] matches one character, a or b, and * matches anything after it, so all names starting with a or b are printed.
`echo *.xyz`	Will show all files with the .xyz extension	`*.xyz`	No file matches the pattern, so bash leaves the pattern unchanged and echo prints it literally.
`ls alfa beta.txt`	Won't show anything	`ls: cannot access 'alfa': No such file or directory ls: cannot access 'beta.txt': No such file or directory`	Word splitting turns the name into two arguments, alfa and beta.txt, and neither file exists, so ls prints two errors.
`ls "alfa beta.txt"`	Will show the file that we have	`'alfa beta.txt'`	Matches the prediction; the quotes keep the name as one argument (ls displays it in quotes because it contains a space).
`echo '$HOME' "$HOME" \$HOME`	Did not know	`$HOME /home/vboxuser $HOME`	Single quotes and the backslash prevent expansion; double quotes allow $HOME to be replaced with its value.

Command	My prediction	Real output	Explanation (if different)
`echo {1..5} ; echo $((7 * 6))`	Will count from 1 to 5 and compute 7*6	`1 2 3 4 5` `42`	Matches the prediction (brace expansion and arithmetic expansion).
`echo "today is $(date +%A), I am $(whoami)"`	Will write this text in the terminal, with the day and the user name	`today is Monday, I am vboxuser`	Matches the prediction; each $(...) is replaced with the output of the command inside it.
`touch log{1..3}.txt && ls log*`	Did not know	`log1.txt log2.txt log3.txt`	Brace expansion creates three file names for touch; because touch succeeds, && runs ls, and log* matches the three new files.

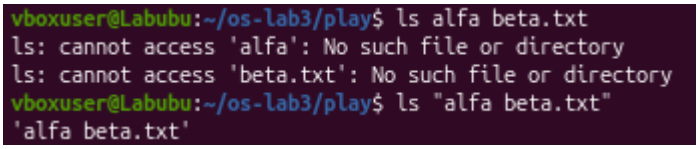


Figure 2.3 – The ls error on a name with a space, and the fix



Figure 2.4 – Single quotes, double quotes and backslash

2.3 Answers to the questions

1. Prediction, real output and explanation for each line

The predictions, the real output and the explanations are given in table 2.1.

2. echo knows nothing about files, yet echo *.c printed file names. Who produced them?

The file names were not produced by `echo`. The shell (bash) expanded `*.c` into the list of matching file names before running `echo`, so `echo` only received the arguments `a.c` and `b.c` and printed them.

3. Why did ls alfa beta.txt fail? Give two different ways to fix it.

`ls alfa beta.txt` failed because the real file name is `alfa beta.txt`, with a space. The shell split the line at the space into two arguments, `alfa` and `beta.txt`, and no files with these names exist. It can be fixed in two ways:

```
ls "alfa beta.txt"
ls alfa\ beta.txt
```

4. What is the difference between single and double quotes?

Single quotes prevent all expansion: `'$HOME'` stays as the text `$HOME`. Double quotes allow expansions such as variables and command substitution, so `"$HOME"` becomes `/home/vboxuser`, while still keeping the result as a single word.

Note: `ls` shows the file with a space as `'alfa beta.txt'` because, when writing to a terminal, it quotes names that contain special characters such as spaces. This way the displayed name can be copied and pasted back into the shell as a single, correct argument.

Figure 2.2 (globbing results) was not saved in the virtual machine, which is no longer available, so the four globbing lines of table 2.1 were run again on my current laptop (Fedora 44), in the same `play` folder with the same files. The results are the same as in table 2.1.

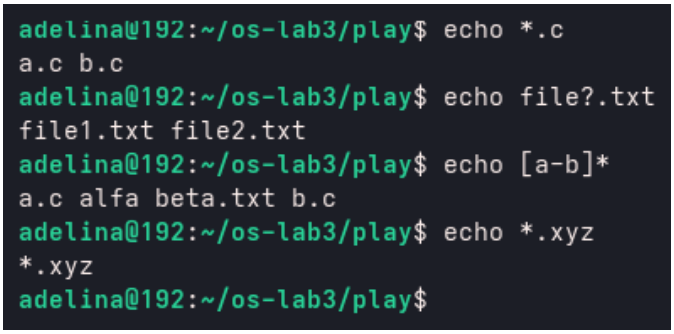


Figure 2.2 – Globbing results (re-run on Fedora 44)

PART 3. REDIRECTION, CHAINING AND JOBS

3.1 Commands executed

The following commands were run one by one and their output was noted:

```
cd ~/os-lab3
ls /etc /nope > out.txt 2> err.txt
wc -l out.txt ; cat err.txt
ls /etc /nope > all.txt 2>&1 ; tail -2 all.txt
echo first > notes.md ; echo second >> notes.md ; cat notes.md
wc -l < access.log
ls /nope 2> /dev/null || echo "failed with code $?"
mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3
sleep 100 &
jobs ; kill %1 ; jobs
```

```
vboxuser@Labubu:~/os-lab3$ cd ~/os-lab3
ls /etc /nope > out.txt 2> err.txt
wc -l out.txt ; cat err.txt
ls /etc /nope > all.txt 2>&1 ; tail -2 all.txt
echo first > notes.md ; echo second >> notes.md ; cat notes.md
wc -l < access.log
ls /nope 2> /dev/null || echo "failed with code $?"
mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3
sleep 100 &
jobs ; kill %1 ; jobs
237 out.txt
ls: cannot access '/nope': No such file or directory
xml
zsh_command_not_found
first
second
500
failed with code 2
I am in /home/vboxuser/os-lab3/tmp
[1] 28369
[1]+  Running                 sleep 100 &
[1]+  Running                 sleep 100 &
[1]+  Terminated              sleep 100
vboxuser@Labubu:~/os-lab3$ ls /etc /nope > all.txt 2>&1
ls /etc /nope 2>&1 > all.txt
ls: cannot access '/nope': No such file or directory
vboxuser@Labubu:~/os-lab3$
```

Figure 3.1 – stdout and stderr in separate files

To compare the order of redirections, the following two versions were run:

```
ls /etc /nope > all.txt 2>&1
ls /etc /nope 2>&1 > all.txt
```

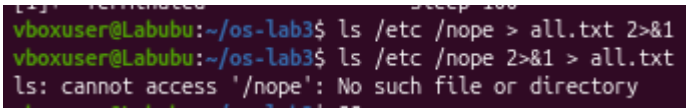


Figure 3.2 – Comparing the order of redirections

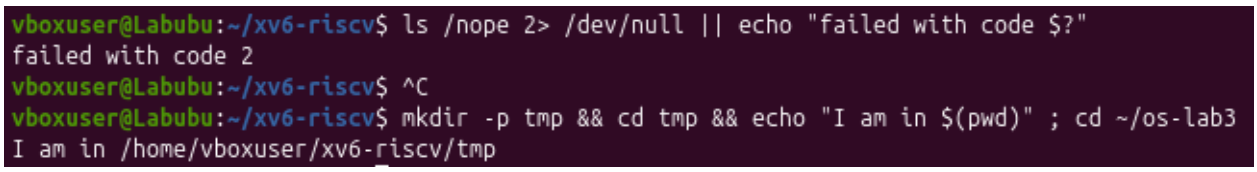


Figure 3.3 – The ; && || operators

Figure 3.3 was taken in a terminal that was open in `~/xv6-riscv`, so `pwd` shows that folder. The job sequence is at the end of Figure 3.1: `sleep 100 &` printed `[1] 28369`, `jobs` showed it Running, and after `kill %1` the second `jobs` showed Terminated. Figure 3.4 is a later re-run with no job left, so kill printed `no such job`.

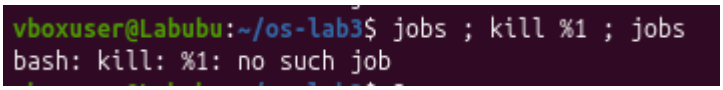


Figure 3.4 – Running a background job and stopping it

3.2 Answers to the questions

1. Which file got the error message, and which got the listing?

With `ls /etc /nope > out.txt 2> err.txt`, the file `out.txt` got the normal output, i.e. the listing of `/etc`, and the file `err.txt` got the error message about `/nope`. Standard output (stdout) went to `out.txt`, while standard error (stderr) went to `err.txt`.

2. What is the difference between > all.txt 2>&1 and 2>&1 > all.txt?

With `ls /etc /nope > all.txt 2>&1`, both the listing and the error message go into `all.txt`. Redirections are processed from left to right: `> all.txt` first sends stdout to `all.txt`, and then `2>&1` makes stderr go wherever stdout currently goes, which is `all.txt`.

With `ls /etc /nope 2>&1 > all.txt`, the error appears in the terminal and only the listing goes into `all.txt`. Here `2>&1` first makes stderr point to the terminal, where stdout points at that moment, and only after that `> all.txt` redirects stdout alone to the file, so stderr stays connected to the terminal.

Because redirections are evaluated from left to right, changing their order changes the result.

3. Why does wc -l < access.log print only a number?

In `wc -l < access.log`, the shell opens `access.log` and connects it to the standard input of `wc`. Since `wc` receives the data through stdin and is not given a file name as an argument, it does not know the file name and prints only the number: `500`.

In `wc -l access.log`, the name is passed explicitly as an argument, so `wc` opens the file itself and prints both the line count and the file name: `500 access.log`.

4. Explain each operator in one sentence: ; && ||

- `;` runs the next command regardless of whether the previous command succeeded or failed.
- `&&` runs the next command only if the previous command succeeded (exit status 0).
- `||` runs the next command only if the previous command failed (non-zero exit status).

PART 4. ANSWER QUESTIONS WITH ONE-LINERS

Each line of `access.log` contains a date, a time, a user, a result (OK or FAIL) and a page, for example `2026-10-05 10:01 user1 OK /page1`. Every question was answered with one command line, and all lines were saved in `oneliners.sh`, each preceded by the question as a comment.

Q1. How many requests failed?

```
grep -c 'FAIL' access.log
```

Output:

```
100
```

There are 100 failed requests.

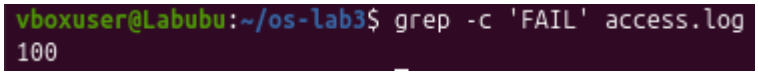


Figure 4.1 – Number of failed requests

Q2. How many different pages were requested, and which?

```
cut -d' ' -f5 access.log | sort -u | uniq -c
```

Output:

```
      1 /page0
      1 /page1
      1 /page2
      1 /page3
```

Four different pages were requested: /page0, /page1, /page2 and /page3.

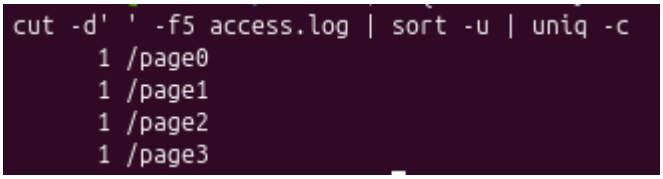


Figure 4.2 – Distinct pages requested

Q3. How many requests did each page get?

```
cut -d' ' -f5 access.log | sort | uniq -c | sort -rn
```

Output:

```
    125 /page3
    125 /page2
    125 /page1
    125 /page0
```

Each page received 125 requests; the counts add up to 500, the total number of lines in the log.

```
vboxuser@Labubu:~/os-lab3$ cut -d' ' -f5 access.log | sort | uniq -c | sort -rn
    125 /page3
    125 /page2
    125 /page1
    125 /page0
```

Figure 4.3 – Requests per page

Q4. Which user or users have the most failed requests, and how many?

```
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn
```

Output (Figure 4.4): `15 user5`, `15 user3`, then `14` for each of user6, user4, user2, user1 and user0. The list is sorted from the largest count, so the first lines answer the question: user3 and user5 have the most failed requests, 15 each. The seven counts add up to 100, the answer of Q1.

```
vboxuser@Labubu:~/os-lab3$ grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn
     15 user5
     15 user3
     14 user6
     14 user4
     14 user2
     14 user1
     14 user0
```

Figure 4.4 – Users with the most failed requests

Q5. How many requests did user3 make, and how many of them failed?

```
grep 'user3' access.log | cut -d' ' -f4 | sort | uniq -c
```

Output:

```
15 FAIL
57 OK
```

user3 made 15 + 57 = 72 requests, of which 15 failed.

```
vboxuser@Labubu:~/os-lab3$ grep 'user3' access.log | cut -d' ' -f4 | sort | uniq -c
     15 FAIL
     57 OK
```

Figure 4.5 – Requests and failures of user3

Q6. Print the last 3 failed requests, showing only time and user.

```
grep FAIL access.log | tail -3 | cut -d' ' -f2,3
```

Output:

```
10:10 user0
10:15 user5
10:20 user3
```

```
vboxuser@Labubu:~/os-lab3$ grep FAIL access.log | tail -3 | cut -d' ' -f2,3
10:10 user0
10:15 user5
10:20 user3
```

Figure 4.6 – Last three failed requests

Q7. Which login shells appear in /etc/passwd, and how many accounts use each?

```
cut -d: -f7 /etc/passwd | sort | uniq -c
```

Output (Figure 4.7): `2 /bin/bash`, `6 /bin/false`, `1 /bin/sync` and `40 /usr/sbin/nologin`. Four shells appear; 40 system accounts cannot log in at all, and only 2 accounts have a real login shell, /bin/bash.

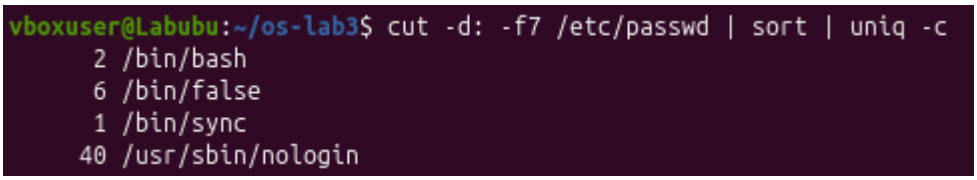


Figure 4.7 – Login shells in /etc/passwd

Q8. Why do ls /etc | wc -l and ls -l /etc | wc -l differ by exactly one?

```
ls /etc | wc -l ; ls -l /etc | wc -l
```

Output:

236 for `ls /etc` and 237 for `ls -l /etc` (Figure 4.8).

Prediction: the long format adds one extra line. Explanation: `ls` prints one name per line when its output goes to a pipe, while `ls -l` prints the same entries plus a first line `total N`, which shows the total number of disk blocks used by the listed files. That header line is the difference of exactly one.

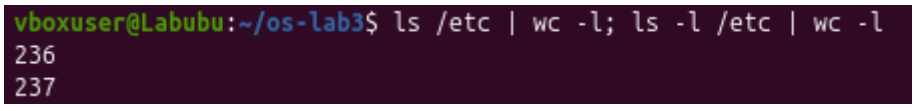


Figure 4.8 – Comparing ls and ls -l line counts

Bonus: failed requests in minutes ending in 0

```
grep -E ' 10:[0-5]0 .*FAIL' access.log | wc -l
```

Output: `50`. The pattern `10:[0-5]0` matches only the minutes 00, 10, 20, 30, 40 and 50, and `.*FAIL` keeps only failed requests, so there are 50 such lines.

Full run of oneliners.sh

The complete file was run with `bash oneliners.sh` inside `~/os-lab3`.



Figure 4.9 – Full run of oneliners.sh

PART 5. YOUR FIRST SCRIPT

5.1 The report.sh script

The script `report.sh` was created in `~/os-lab3`, and the TODO was completed with a loop that prints the number of failures for each user from user0 to user6:

```
#!/bin/bash
# usage: ./report.sh access.log
log=$1
if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi
total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"
for n in $(seq 0 6); do
    echo "user$n: $(grep " user$n " "$log" | grep -c FAIL) failures"
done
```



Figure 5.1 – report.sh being written in nano, before the TODO loop was added

5.2 Running the script

```
chmod +x report.sh
./report.sh access.log
./report.sh nope.log ; echo "exit code: $?"
```

The output was:

```
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
no such file: nope.log
exit code: 1
```

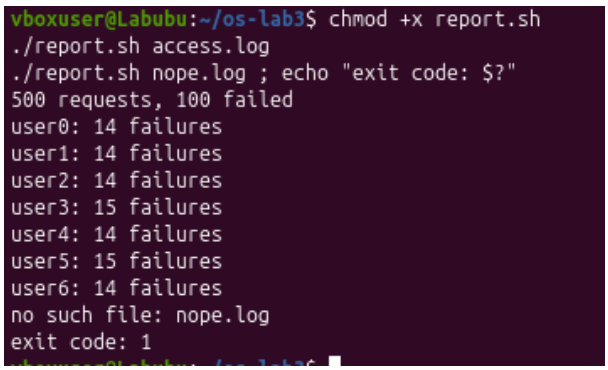


Figure 5.2 – Error handling for a missing file

5.3 Answers to the questions

1. What does #!/bin/bash do, and what happens before chmod +x?

The first line `#!/bin/bash` is called a shebang. It tells Linux to run the script with the Bash interpreter, which is why the script can be started as `./report.sh access.log` instead of `bash report.sh access.log`.

If the file is created and run with `./report.sh access.log` immediately, the shell reports `Permission denied`, because the file does not have the execute permission. After `chmod +x report.sh`, it can be executed directly.

2. Why must you type ./report.sh and not just report.sh?

When only `report.sh` is typed, the shell searches for a program with that name in the directories listed in `$PATH`. The current directory is usually not included in `PATH`, for security reasons, so the script is not found. The prefix `./` means "look in the current directory", so `./report.sh` points directly to the file `report.sh` in the current directory. The search path can be displayed with `echo "$PATH"`.

3. Why does the script write the error with >&2 and end with exit 1?

The line `echo "no such file: $log" >&2` sends the error message to standard error instead of standard output. Normally `echo` writes to stdout, but error messages should go to stderr. This is useful when the output is redirected: with `./report.sh nope.log > output.txt`, the normal output goes into `output.txt`, while the error still appears in the terminal.

`exit 1` stops the script and tells the operating system that the script failed: 0 normally means success, and non-zero values mean some kind of error. That is why `./report.sh nope.log ; echo "exit code: $?"` prints `exit code: 1`, where `$?` holds the exit code of the previous command.

CONCLUSION

In this laboratory work, the bash command line was studied as a tool for working quickly and for processing data.

In the first part, Tab completion and Ctrl+R history search were used to speed up typing, and the documentation tools `man`, `help` and `--help` were compared. It was shown that `cd` and `echo` are shell built-ins while `ls` is an external program in `/usr/bin`, that `cd` must be a built-in in order to change the directory of the shell itself, and that a missing command returns the exit code 127.

In the second part, the predictions made before running each command showed that the shell, not the program, performs globbing, brace, arithmetic and command expansion, and that quoting controls which of these expansions take place; a file name containing a space must be quoted or escaped.

In the third part, stdout and stderr were redirected to separate files and to the same file, and it was observed that redirections are processed from left to right, so their order matters. The operators `;`, `&&` and `||` and background jobs were also used.

In the fourth part, questions about a 500-line access log were answered with one-line pipelines: 100 requests failed, the four pages received 125 requests each, and user3 and user5 had the most failures (15 each). Finally, in the fifth part, these pipelines were turned into a script that checks its argument, reports errors on stderr and returns a non-zero exit code on failure.

REFERENCES

1. bash(1) – Linux manual page. [online]. Available: https://man7.org/linux/man-pages/man1/bash.1.html
2. GNU Bash Reference Manual. Free Software Foundation. [online]. Available: https://www.gnu.org/software/bash/manual/bash.html
3. GNU Coreutils Manual. Free Software Foundation. [online]. Available: https://www.gnu.org/software/coreutils/manual/coreutils.html
4. Shotts W. The Linux Command Line. 2nd ed. No Starch Press, 2019. [online]. Available: https://linuxcommand.org/tlcl.php